

Value-at-Risk sur les indices boursiers MENA

GARCH, Machine Learning & Deep Learning

Rapport explicatif détaillé, ligne par ligne, du notebook autonome
notebook/VaR_MENA_complet.ipynb

*Comparaison de 7 modèles de prévision (ARIMA, SARIMA, GARCH, Random Forest,
XGBoost, ANN, LSTM) pour le calcul et le backtesting de la Value-at-Risk (VaR)
sur 4 indices boursiers du Moyen-Orient / Afrique du Nord :
Tunindex, ADI, MASI, TASI*

Document destiné à la soutenance orale du projet.

Décomposition, stationnarité, volatilité

Table des matieres

0. Présentation du projet et méthodologie générale	4
1. Chargement et prétraitement des données	6
2. Analyse exploratoire et décomposition	9
3. Stationnarité : définitions et tests	11
4. Modèles classiques : ARIMA et SARIMA	13
5. Volatilité : le modèle GARCH	16
6. Machine Learning : Random Forest et XGBoost	18
7. Deep Learning : ANN et LSTM	20
8. La VaR par Bootstrap Historical Simulation (BHS)	22
9. Backtesting : Kupiec, Christoffersen, zones Bâle	23
10. Comparaison globale (4 indices x 7 modèles) et conclusion	26

0. Présentation du projet et méthodologie générale

À quoi sert ce rapport ?

Ce document explique, en français et pas à pas, le notebook autonome `notebook/VaR_MENA_complet.ipynb`. Ce notebook ne dépend d'aucun code externe : toutes les fonctions (nettoyage des données, tests statistiques, modèles de prévision, calcul de la VaR, backtesting) sont écrites directement dans ses cellules de code. L'objectif de ce rapport est de vous permettre de **défendre** ce travail à l'oral : pour chaque cellule de code, on explique ce qu'elle fait, pourquoi elle le fait, et ce que les résultats obtenus signifient concrètement.

Contexte : les indices boursiers MENA

Les marchés boursiers de la région **MENA** (Moyen-Orient et Afrique du Nord) sont réputés moins liquides et plus volatils que les marchés développés (chocs pétroliers, instabilité politique, faible profondeur de marché). Le projet compare **4 indices MENA** :

- **Tunindex** (Tunisie)
- **ADI** (Abu Dhabi - ADX General Index)
- **MASI** (Maroc - Moroccan All Shares Index)
- **TASI** (Arabie Saoudite - Tadawul All Share Index)

Deux indices de marchés développés servent de simples **benchmarks de contexte** (pour comparer les niveaux de volatilité), sans faire l'objet du backtesting VaR (pas de fichier de test dédié pour eux) : **CAC 40** (France) et **S&P 500** (États-Unis).

Qu'est-ce que la Value-at-Risk (VaR) ?

La VaR à un horizon d'un jour et à un seuil de confiance $1 - \alpha$ est le rendement seuil tel que la probabilité de subir une perte plus grande ne dépasse pas α . Concrètement, une VaR à 95% de -2% signifie : "il y a seulement 5% de chances que la perte demain dépasse 2%". Plus α est petit (1% versus 5%), plus on regarde loin dans la queue de distribution des pertes, donc plus la VaR est sévère (plus négative).

Dans ce projet, la VaR est calculée par une méthode appelée **Bootstrap Historical Simulation (BHS)** : pour chaque modèle, on combine une prévision de tendance (μ_t), une prévision de volatilité (σ_t), et la forme empirique (non-normale) des erreurs de prévision passées (résidus standardisés), rééchantillonnée par bootstrap. La formule générale est :

$$VaR_t(\alpha) = \mu_t + \sigma_t * Q_{\alpha}(\text{bootstrap}(\text{residus_standardises}))$$

Ce cadre unifié permet de comparer des modèles très différents (ARIMA, GARCH, Random Forest, LSTM...) avec exactement la même formule finale de VaR : seuls μ_t , σ_t et le pool de résidus changent d'un modèle à l'autre.

La méthodologie « train-once / walk-forward » (sans réentraînement)

Point méthodologique clé à bien comprendre et à savoir expliquer à l'oral : chaque modèle est entraîné **un e seule fois** sur l'échantillon d'entraînement (train). Ensuite, on avance **pas à pas** (jour par jour) sur l'échantillon de test :

- à chaque jour t du test, le modèle produit une prévision pour le jour t à partir de ce qu'il connaît ;
- on injecte ensuite le rendement **réellement observé** ce jour-là dans la fenêtre d'entrée du modèle (pour pouvoir prévoir le jour $t+1$) ;

- mais on **ne ré-estime jamais** les paramètres du modèle sur le test (pas de nouveau fit).

C'est un compromis réaliste entre (a) un modèle figé qui ignore complètement le test set (trop optimiste : le modèle « voit » un futur qu'il ne pourrait pas connaître en pratique s'il changeait de régime), et (b) un modèle réentraîné à chaque pas (coûteux en temps de calcul, et peu réaliste en production où l'on ne recalibre pas un modèle tous les jours). Cette approche s'appelle le **walk-forward** ou **prévision à un pas (one-step-ahead)** avec **train unique**.

L'hypothèse testée par le projet

L'hypothèse de départ, formulée avant tout calcul, est la suivante :

« Le LSTM surpasse les autres modèles sur l'indice ADI »

ADI (Abu Dhabi) a été choisi comme indice-test de cette hypothèse car son profil de volatilité (marché relativement liquide mais toujours émergent) était jugé a priori propice à un modèle non-linéaire capable d'apprendre des dépendances temporelles complexes. Le notebook teste cette hypothèse en conditions réelles, avec les **7 modèles** suivants : ARIMA, SARIMA, GARCH, Random Forest, XGBoost, ANN, LSTM. **Attention** : la conclusion honnête de ce projet (chapitre 10) est que cette hypothèse n'est **pas confirmée** - le LSTM est fiable mais pas le meilleur modèle. Il faut savoir l'assumer à l'oral plutôt que de le cacher : c'est un résultat scientifique valable.

Plan du notebook (et de ce rapport)

- 0. Initialisation (imports, chemin des données)
- 1. Introduction et objectifs
- 2. Chargement et prétraitement des 6 indices
- 3. Analyse exploratoire et décomposition
- 4. Stationnarité (ADF, KPSS, ACF/PACF)
- 5. Modèles classiques ARIMA / SARIMA (exécution en direct sur ADI)
- 6. Volatilité GARCH (exécution en direct sur ADI)
- 7. Machine Learning : Random Forest et XGBoost (exécution en direct sur ADI)
- 8. Deep Learning : ANN et LSTM (exécution en direct sur ADI)
- 9. VaR par Bootstrap Historical Simulation (BHS)
- 10. Backtesting : Kupiec, Christoffersen, zones Bâle
- 11. Comparaison globale (4 indices x 7 modèles) et conclusion

À retenir

Le notebook exécute réellement l'intégralité du pipeline : les 7 modèles, sur les 4 indices MENA, aux 2 seuils alpha (5% et 1%), y compris pour la comparaison globale finale. Il ne relit aucun fichier de résultats pré-calculé : tout est recalculé à l'exécution. Seul compromis assumé : les réseaux de neurones (ANN, LSTM) sont entraînés avec 20 époques (au lieu de 30 dans le run de référence du package source), pour que le notebook s'exécute en un temps raisonnable.

1. Chargement et prétraitement des données

Pourquoi nettoyer les données brutes ?

Les fichiers CSV bruts (exportés d'un site financier) contiennent des formats non numériques directement exploitables : dates au format texte (Jan 04, 2005), nombres avec séparateurs de milliers (1,234.5), volumes suffixés par M (millions) ou K (milliers), pourcentages avec le symbole %. Il faut les convertir en types numériques/temporels propres avant tout calcul.

Le code (cellule 2 - initialisation)

```
%matplotlib inline
import warnings
warnings.filterwarnings("ignore")

import pathlib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

SEED = 42

root = pathlib.Path.cwd().resolve()
for c in (root,*root.parents):
    if (c / "data (1)" / "data").is_dir():
        DATA_DIR = c / "data (1)" / "data"
        break
else:
    raise RuntimeError("data (1)/data not found starting from " + str(root))
```

Explication bloc par bloc

- `%matplotlib inline` : commande Jupyter qui affiche les graphiques directement sous la cellule.
- `warnings.filterwarnings("ignore")` : masque les avertissements bénins (convergence, interpolation) qui seraient sinon affichés à chaque modèle - ce n'est pas une erreur cachée, juste du bruit attendu.
- `SEED = 42` : graine aléatoire fixée une fois pour toutes, pour que le bootstrap, les Random Forest/XGBoost et les réseaux de neurones donnent des résultats **reproductibles** d'une exécution à l'autre.
- La boucle `for c in (root, *root.parents)` remonte l'arborescence des dossiers depuis le répertoire courant du kernel jusqu'à trouver `data (1)/data` : cela rend le notebook exécutable quel que soit l'endroit d'où Jupyter est lancé, sans chemin absolu codé en dur.

Le code (cellule 5 - fonctions de chargement)

```
def _num(s):
    return pd.to_numeric(s.astype(str).str.replace(",", "", regex=False), errors="coerce")

def _vol(s):
    s = s.astype(str).str.strip()
    mult = np.where(s.str.endswith("M"), 1e6, np.where(s.str.endswith("K"), 1e3, 1.0))
    base = pd.to_numeric(s.str.replace("[MK]", "", regex=True), errors="coerce")
    return base * mult

def load_index(csv_path):
    df = pd.read_csv(csv_path)
    df.columns = [c.strip().strip("'") for c in df.columns]
    df["Date"] = pd.to_datetime(df["Date"], format="%b %d, %Y")
    for c in ["Price", "Open", "High", "Low"]:
        df[c] = _num(df[c])
    df["Volume"] = _vol(df["Vol."])
    df["ChangePct"] = _num(df["Change %"].astype(str).str.replace("%", "", regex=False))
    df = df.set_index("Date").sort_index()
    return df[["Price", "Open", "High", "Low", "Volume", "ChangePct"]]
```

```
def log_returns(prices):
    return (100 * np.log(prices / prices.shift(1))).dropna()
```

Explication bloc par bloc

- `_num(s)` : convertit une colonne texte en nombre, en retirant d'abord les virgules de séparation de milliers (1,234 -> 1234). `errors="coerce"` transforme toute valeur non convertible (ex : -) en NaN plutôt que de faire planter le programme.
- `_vol(s)` : gère le suffixe des volumes (1.2M -> 1 200 000, 350K -> 350 000) via un multiplicateur (`np.where` imbriqué), puis retire la lettre et convertit le nombre restant.
- `load_index` : lit le CSV, nettoie les noms de colonnes (espaces, guillemets résiduels), parse la date au format %b %d, %Y (ex : Jan 04, 2005), nettoie les colonnes de prix et le volume, trie chronologiquement par date (`sort_index`), et ne garde que les colonnes utiles.
- `log_returns(prices)` : calcule le **rendement logarithmique en pourcentage** : $r_t = 100 * \ln(P_t / P_{t-1})$. Le `.dropna()` retire la première valeur (non définie car il n'existe pas de prix à t-1 avant le début de la série).

Pourquoi le rendement logarithmique plutôt que le prix brut ?

C'est une question quasi certaine à l'oral. Trois raisons principales :

- **Stationnarité** : le prix d'un indice a une tendance (il monte ou baisse sur le long terme), sa moyenne et sa variance ne sont pas constantes dans le temps -> non-stationnaire. Le rendement log, lui, oscille autour d'une moyenne quasi nulle et stable -> (approximativement) stationnaire, ce qui est **requis** par la quasi-totalité des modèles de séries temporelles (ARIMA, GARCH...).
- **Additivité temporelle** : la somme de rendements log sur plusieurs jours correspond exactement au rendement log cumulé ($\ln(P_t/P_0) = \text{somme des } r_i$), ce qui n'est vrai qu'approximativement pour des rendements simples ($P_t/P_{t-1} - 1$).
- **Symétrie et stabilisation de la variance** : le log rend symétrique le traitement d'une hausse de 10% et d'une baisse de 10% (qui ne sont pas des variations symétriques en rendement simple), et atténue l'effet des grandes variations de niveau de prix.

Chargement de tous les indices et aperçu statistique (cellule 5, suite)

```
INDEX_FILES = {"ADI": "ADI.csv", "CAC40": "CAC40.csv", "MASI": "MASI.csv",
               "S&P500": "S&P500.csv", "TASI": "TASI.csv", "Tunindex": "Tunindex.csv"}
MENA = ["Tunindex", "ADI", "MASI", "TASI"]
BENCHMARKS = ["CAC40", "S&P500"]
ALL_INDICES = MENA + BENCHMARKS

prices, returns = {}, {}
for name in ALL_INDICES:
    df = load_index(DATA_DIR / INDEX_FILES[name])
    prices[name] = df["Price"]
    returns[name] = log_returns(df["Price"])
```

Chaque indice est chargé et son prix + son rendement log sont stockés dans deux dictionnaires Python (`prices`, `returns`), indexés par nom d'indice. Le notebook calcule ensuite un tableau récapitulatif (aperçu) avec, pour chaque indice, le nombre d'observations, les dates de début/fin, la moyenne, l'écart-type, le min/max, le skew (asymétrie) et la kurtosis (aplatissement) des rendements.

Indice	n obs	Début	Fin	Moy(%)	Écart-t(%)	Min(%)	Max(%)	Skew	Kurt.
Tunindex	2470	2005-01-04	2014-12-31	0.0542	0.5868	-5.00	4.11	-0.54	11.64
ADI	2584	2005-01-04	2014-12-31	0.0131	1.2701	-8.68	7.63	-0.06	6.99
MASI	2495	2005-01-04	2014-12-31	0.0303	0.8302	-5.02	4.46	-0.39	4.89
TASI	2577	2005-01-04	2014-12-31	0.0009	1.6852	-10.33	9.39	-0.90	8.22
CAC40	2559	2005-01-04	2014-12-31	0.0040	1.4468	-9.47	10.59	0.05	6.78

Indice	n obs	Début	Fin	Moy(%)	Écart-t(%)	Min(%)	Max(%)	Skew	Kurt.
S&P500	2535	2005-01-04	2014-12-31	0.0210	1.2974	-10.40	13.20	-0.12	14.10

Interprétation du tableau (résultat réel du notebook)

Les 4 indices MENA affichent une volatilité (écart-type) du même ordre de grandeur, voire supérieure, à celle des benchmarks développés : TASI (1.6852%) et ADI (1.2701%) sont plus volatils que le CAC40 (1.4468%) et proches du S&P500 (1.2974%). Toutes les séries ont une kurtosis très supérieure à 3 (celle d'une loi normale) - de 4.89 (MASI) à 14.1 (S&P500) - signe de queues de distribution épaisses : les événements extrêmes sont plus fréquents que sous une hypothèse de normalité. C'est un des « faits stylisés » classiques des rendements financiers, qui justifie de ne pas supposer une loi normale pour la VaR (d'où le choix du bootstrap, chapitre 8). Le skew est majoritairement négatif (Tunindex -0.54, TASI -0.90), signe que les baisses extrêmes sont plus marquées que les hausses extrêmes - typique des marchés actions.

Rôle des benchmarks : CAC 40 et S&P 500 ne servent que de repère de contexte. Comme il n'existe pas de fichier de test dédié pour eux (*Test.csv), ils ne font l'objet d'aucun backtesting VaR : les chapitres 8 à 10 portent uniquement sur les 4 indices MENA.

2. Analyse exploratoire et décomposition

Objectif de ce chapitre

Avant de modéliser, on regarde « à l'œil » la série de rendements ADI et on illustre deux notions fondamentales du cours de séries temporelles : la **décomposition** (tendance / saisonnalité / résidu) et le **clustering de volatilité** (regroupement des périodes de forte variance).

Graphique des rendements (cellule 8)

```
fig, ax = plt.subplots(figsize=(10, 3))
ax.plot(returns["ADI"].index, returns["ADI"].values, lw=.7)
ax.set_title("Rendements log journaliers - ADI (%)")
plt.tight_layout()
plt.show()
```

Simple tracé de la série temporelle des rendements ADI. On y observe déjà à l'œil des périodes de plus grande amplitude de variation (2008-2009, crise financière mondiale) alternant avec des périodes plus calmes.

Décomposition tendance / saisonnalité / résidu (cellule 9)

Un modèle de décomposition additif s'écrit :

$$Y_t = T_t + S_t + e_t \quad (\text{Tendance} + \text{Saisonnalité} + \text{résidu/erreur})$$

```
from statsmodels.tsa.seasonal import seasonal_decompose

decomp = seasonal_decompose(prices["ADI"], period=5, model="additive", extrapolate_trend="freq")
fig = decomp.plot()
```

Explication

- `seasonal_decompose` estime T_t (tendance, par moyenne mobile), puis S_t (motif saisonnier moyen répété toutes les `period` observations) et enfin le résidu $e_t = Y_t - T_t - S_t$.
- `period=5` correspond à une semaine de 5 jours ouvrés : c'est un choix **illustratif** - un indice boursier n'a pas de vraie saisonnalité « physique » comme des ventes ou de la météo. La décomposition reste utile ici comme outil pédagogique pour isoler visuellement la tendance de long terme du prix.
- `model="additive"` suppose que les composantes s'additionnent (plutôt que se multiplient, ce qui serait le cas si l'amplitude saisonnière croissait avec le niveau de la série).
- `extrapolate_trend="freq"` évite d'avoir des NaN en début/fin de série (la moyenne mobile de la tendance ne peut normalement pas être calculée sur les tout premiers/derniers points).

Clustering de volatilité (cellule 10)

```
fig, ax = plt.subplots(figsize=(10, 3))
returns["ADI"].rolling(20).std().plot(ax=ax, color="darkorange")
ax.set_title("Volatilité glissante (fenêtre 20j) - rendements ADI (%)")
```

`rolling(20).std()` calcule l'écart-type des rendements sur une fenêtre glissante de 20 jours (environ un mois boursier). Le graphique obtenu montre des **périodes de volatilité groupées** : des phases de forte variance suivies d'autres phases de forte variance, plutôt qu'une variance dispersée au hasard dans le temps.

Interprétation

Ce clustering de volatilité est précisément ce qu'un modèle à variance constante (comme un ARIMA classique) ne peut PAS capturer, et ce que le modèle GARCH (chapitre 5) est spécifiquement conçu pour modéliser : la variance d'aujourd'hui dépend de la variance et des chocs d'hier. C'est l'argument central qui justifie l'usage du GARCH plutôt qu'une hypothèse de variance constante pour des données financières.

3. Stationnarité : définitions et tests

Qu'est-ce que la stationnarité et pourquoi est-ce requis ?

Une série temporelle est dite (faiblement / au second ordre) **stationnaire** si ses propriétés statistiques ne dépendent pas du temps :

- sa **moyenne** $E[Y_t]$ est constante dans le temps ;
- sa **variance** $\text{Var}(Y_t)$ est constante dans le temps ;
- son **autocovariance** $\text{Cov}(Y_t, Y_{t+k})$ ne dépend que du décalage k , pas de l'instant t .

C'est une hypothèse requise par la quasi-totalité des modèles classiques (ARIMA, GARCH...) : leurs paramètres (moyenne, variance, coefficients d'autocorrélation) sont supposés **constants** dans le temps - si la série n'est pas stationnaire, ces paramètres estimés sur une période ne seraient plus valables sur une autre, et les propriétés statistiques des tests (p-values, intervalles de confiance) ne seraient plus fiables.

Deux tests complémentaires : ADF et KPSS

Les deux tests ont des hypothèses nulles **opposées** : on les combine pour une conclusion robuste.

Test ADF (Augmented Dickey-Fuller)

- **H0** : la série possède une racine unitaire -> elle est **non-stationnaire**.
- **H1** : la série est stationnaire.
- Règle de lecture : si $p < 0.05$, on **rejette H0** -> la série est jugée stationnaire.

Test KPSS (Kwiatkowski-Phillips-Schmidt-Shin)

- **H0** : la série est **stationnaire**.
- **H1** : la série n'est pas stationnaire (racine unitaire).
- Règle de lecture : si $p > 0.05$, on **ne rejette pas H0** -> la série est jugée stationnaire.

Combiner les deux est utile car ils peuvent parfois se contredire (l'un rejette, l'autre pas) : quand ADF conclut à la stationnarité ET que KPSS ne la rejette pas, la conclusion est robuste.

Le code (cellule 13)

```
def adf_test(series):
    stat, p, *_ = adfuller(series.dropna(), autolag="AIC")
    return {"stat": stat, "pvalue": p, "stationary": p < 0.05}

def kpss_test(series):
    with warnings.catch_warnings():
        warnings.filterwarnings("ignore", category=InterpolationWarning)
        stat, p, *_ = kpss(series.dropna(), regression="c", nlags="auto")
    return {"stat": stat, "pvalue": p, "stationary": p > 0.05}

adf_price, kpss_price = adf_test(prices["ADI"]), kpss_test(prices["ADI"])
adf_ret, kpss_ret = adf_test(returns["ADI"]), kpss_test(returns["ADI"])
```

- `adfuller(..., autolag="AIC")` : la fonction `statsmodels` choisit automatiquement le nombre de retards à inclure dans la régression de test en minimisant le critère d'information AIC.
- `kpss(..., regression="c", nlags="auto")` : `regression="c"` teste la stationnarité autour d'une constante (pas d'une tendance linéaire).
- Le `warnings.catch_warnings()` autour de `kpss` masque un avertissement bénin : la table de p-values du test KPSS est bornée, et une série très stationnaire (comme un rendement financier) produit souvent une statistique hors de cette plage - ce n'est pas un bug, juste une limite de précision de la table de référence.
- On applique les deux tests **à la fois** sur le **prix** ADI (niveau) et sur les **rendements** ADI, pour comparer

directement les deux cas.

Série	ADF stat	ADF p-value	ADF station.	KPSS stat	KPSS p-value	KPSS station.
ADI - Prix	-1.634	0.4655	Non	1.780	0.0100	Non
ADI - Rendements	-8.347	0.0000	Oui	0.197	0.1000	Oui

Interprétation (résultat réel du notebook)

Sur le PRIX ADI : ADF p-value = 0.4655 (largement > 0.05 , on ne rejette PAS H_0) -> racine unitaire non rejetée -> non-stationnaire ; KPSS p-value = 0.01 (< 0.05 , on rejette H_0 de stationnarité) -> non-stationnaire. Les deux tests s'accordent : le prix est non-stationnaire, ce qui est normal pour un niveau de prix qui suit une marche aléatoire avec tendance. Sur les RENDEMENTS ADI : ADF p-value = 0.0000 (< 0.05 , on rejette H_0) -> stationnaire ; KPSS p-value = 0.10 (> 0.05 , on ne rejette pas H_0) -> stationnaire. Les deux tests s'accordent de nouveau, cette fois pour conclure à la stationnarité. Conclusion : c'est bien la série de RENDEMENTS (déjà « différenciée » une fois via le log-retour) qui est utilisée pour tous les modèles du notebook, jamais le prix brut.

ACF / PACF : à quoi ça sert (cellule 14)

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
plot_acf(returns["ADI"], lags=20, ax=axes[0])
plot_pacf(returns["ADI"], lags=20, ax=axes[1])
```

L'**ACF** (fonction d'autocorrélation) mesure la corrélation entre Y_t et Y_{t-k} pour différents décalages k , y compris les effets indirects transmis par les retards intermédiaires. La **PACF** (autocorrélation partielle) mesure la même chose mais en retirant l'effet des retards intermédiaires - elle isole la corrélation « directe » à chaque décalage. Ces deux graphiques servent classiquement à identifier les ordres p (AR) et q (MA) d'un modèle ARIMA : un pic isolé en PACF au retard p suggère un terme $AR(p)$; un pic isolé en ACF au retard q suggère un terme $MA(q)$.

Interprétation

Pour les rendements ADI, l'ACF/PACF ne montre pas d'autocorrélation linéaire forte et persistante au-delà du bruit statistique attendu - cohérent avec un marché proche de l'efficience informationnelle en moyenne (les rendements passés n'aident pas beaucoup à prédire le rendement futur). Cela motive l'usage d'ordres ARIMA/SARIMA modestes (chapitre 4). Mais attention : cette absence d'autocorrélation concerne la MOYENNE des rendements - la VOLATILITÉ, elle, reste fortement autocorrélée (chapitres 2 et 5), d'où la nécessité du GARCH.

4. Modèles classiques : ARIMA et SARIMA

Rappel du modèle ARIMA(p,d,q)

- **AR(p)** - AutoRégressif d'ordre p : la valeur au temps t dépend linéairement des p valeurs passées ($Y_t = c + \phi_1 Y_{(t-1)} + \dots + \phi_p Y_{(t-p)} + e_t$).
- **I(d)** - Intégré d'ordre d : nombre de fois qu'il faut différencier la série ($Y_t - Y_{(t-1)}$) pour la rendre stationnaire.
- **MA(q)** - Moyenne Mobile d'ordre q : la valeur au temps t dépend linéairement des q erreurs de prévision passées ($e_{(t-1)}, \dots, e_{(t-q)}$).

Un **SARIMA(p,d,q)(P,D,Q,m)** ajoute une composante saisonnière : les mêmes idées (AR, I, MA) mais appliquées à une périodicité m (ici m=5, un cycle hebdomadaire de jours ouvrés), avec ses propres ordres P, D, Q saisonniers.

L'interface commune ForecastResult et le moteur de VaR (cellule 17)

```
@dataclass
class ForecastResult:
    mu: np.ndarray
    sigma: np.ndarray
    std_resid: np.ndarray
    y_true: np.ndarray
    dates: pd.DatetimeIndex
    name: str

def bhs_quantile(mu, sigma, std_resid, alpha, n_boot=10000, seed=SEED):
    rng = np.random.default_rng(seed)
    boot = rng.choice(std_resid, size=n_boot, replace=True)
    q = np.quantile(boot, alpha)
    return float(mu + sigma * q)

def var_series(fc, alpha, n_boot=10000, seed=SEED):
    return np.array([
        bhs_quantile(fc.mu[t], fc.sigma[t], fc.std_resid, alpha, n_boot, seed + t)
        for t in range(len(fc.mu))
    ])
```

Explication

- ForecastResult est une structure commune (un dataclass, simple conteneur de données) que **tous les modèles** du notebook remplissent de la même façon : prévisions moyennes mu, prévisions d'écart-type sigma, résidus standardisés d'entraînement std_resid, valeurs réellement observées y_true, dates, et nom du modèle. Cette standardisation permet d'appliquer ensuite exactement le même code de VaR et de backtesting à n'importe quel modèle.
- bhs_quantile implémente la formule de VaR par bootstrap : rng.choice(std_resid, size=10000, replace=True) tire 10 000 résidus **avec remise** dans le pool de résidus standardisés observés à l'entraînement (c'est le « bootstrap »), puis np.quantile(boot, alpha) calcule le quantile empirique alpha de cet échantillon rééchantillonné. On multiplie ce quantile par sigma et on ajoute mu pour obtenir le niveau de VaR final.
- var_series applique bhs_quantile à chaque jour t du test (avec une graine différente seed + t à chaque jour, pour la reproductibilité tout en variant le tirage).

Pourquoi un bootstrap plutôt qu'une hypothèse de loi normale ? Le chapitre 1 a montré que les rendements ont une kurtosis très supérieure à celle d'une loi normale (queues épaisses). Utiliser un quantile de loi normale sous-estimerait systématiquement le risque de queue. Le bootstrap, lui, utilise directement la **forme empirique** (non-paramétrique) de la distribution des résidus, sans supposer aucune loi théorique.

Ajustement ARIMA/SARIMA en walk-forward (cellule 18)

```
def _fit(train, order, seasonal_order):
    m = SARIMAX(train, order=order, seasonal_order=seasonal_order or (0, 0, 0, 0),
                 enforce_stationarity=False, enforce_invertibility=False)
    with warnings.catch_warnings():
        warnings.simplefilter("ignore", ConvergenceWarning)
        return m.fit(dispatch=False, maxiter=200, method="lbfgs")

def walk_forward_arima(train, test, order=None, seasonal_order=None):
    if order is None:
        import pmdarima as pm
        order = pm.auto_arima(train.values, seasonal=False, suppress_warnings=True,
                              error_action="ignore").order

    n = len(train)
    train_ri = pd.Series(np.asarray(train.values), index=pd.RangeIndex(n))
    res = _fit(train_ri, order, seasonal_order)

    raw_resid = np.asarray(res.resid)[1:]
    raw_resid = raw_resid[np.isfinite(raw_resid)]
    sigma = float(np.std(raw_resid))
    std_resid = (raw_resid - np.mean(raw_resid)) / sigma

    mu = np.empty(len(test))
    cur = res
    for t in range(len(test)):
        mu[t] = float(cur.forecast(1).iloc[0])
        nxt = pd.Series([test.values[t]], index=pd.RangeIndex(n + t, n + t + 1))
        cur = cur.append(nxt, refit=False) # walk forward, no re-estimation

    return ForecastResult(mu=mu, sigma=np.full(len(test), sigma), std_resid=std_resid,
                          y_true=test.values, dates=test.index, name=...)

def walk_forward_sarima(train, test, m=5):
    return walk_forward_arima(train, test, order=(1, 0, 1), seasonal_order=(1, 0, 1, m))
```

Explication bloc par bloc

- `pm.auto_arima(...)` : recherche automatique (par `pmdarima`) du meilleur ordre (p, d, q) selon un critère d'information (AIC), sans intervention manuelle - c'est l'identification automatisée des ordres qu'on aurait pu faire à la main avec l'ACF/PACF du chapitre 3.
- `train_ri = pd.Series(..., index=pd.RangeIndex(n))` : **détail d'implémentation important** - on réindexe le train avec un simple index entier (0,1,2,...) plutôt que les dates réelles. Raison : train et test proviennent de deux fichiers CSV distincts qui ne s'enchaînent pas forcément jour calendaire pour jour calendaire (jours fériés, week-ends différents selon le pays) ; sans cela, statsmodels essaierait (et échouerait) d'inférer une fréquence calendaire continue. Les dates réelles du test sont réattachées à la fin, dans `ForecastResult.dates`.
- `res.resid[1:]` : on retire le tout premier résidu, souvent un artefact de « rodage » (burn-in) du filtre de Kalman utilisé en interne par SARIMAX, puis on filtre les valeurs non finies (`np.isfinite`) pour garantir un pool de résidus toujours utilisable par le bootstrap.
- **La boucle walk-forward** : à chaque pas t , `cur.forecast(1)` produit la prévision à un jour ($J+1$). Puis `cur.append(nxt, refit=False)` ajoute le rendement **réellement observé** ce jour-là à l'état interne du modèle, **sans réestimer les paramètres** (`refit=False`) - c'est exactement la contrainte « train-once » du chapitre 0.
- `walk_forward_sarima` réutilise `walk_forward_arima` avec des ordres fixes $(1, 0, 1)(1, 0, 1, 5)$ plutôt que la recherche automatique - un choix simple et raisonnable pour illustrer la composante saisonnière hebdomadaire ($m=5$).

Résultats ARIMA / SARIMA sur ADI (sortie réelle)

ADI - train: 2584 obs | test: 170 obs

ARIMA	MAE=0.7088	RMSE=0.9764
SARIMA	MAE=0.7073	RMSE=0.9759

Interprétation

Le MAE (erreur absolue moyenne) et le RMSE (racine de l'erreur quadratique moyenne) de ARIMA et SARIMA sont quasiment identiques (0.7088 vs 0.7073 en MAE) : sur ADI, la composante saisonnière hebdomadaire ajoutée par SARIMA n'apporte quasiment rien - cohérent avec l'ACF/PACF du chapitre 3 qui ne montrait pas de structure autocorrélée forte. Le point clé à retenir pour la suite : ces deux modèles utilisent un sigma constant dans le temps (calculé une fois sur le train) - ils ne modélisent PAS le clustering de volatilité observé au chapitre 2. C'est précisément ce que corrige le GARCH (chapitre 5).

Visualisation de la VaR ARIMA à 95% (cellule 19)

```
var_arima_95 = var_series(fc_arima, 0.05)
plot_var(fc_arima, var_arima_95, title="VaR 95% - ARIMA (BHS) - ADI (test)")
```

Le graphique superpose le rendement réalisé (courbe fine), la VaR (courbe rouge) et les **violations** (points noirs, jours où le rendement réalisé est passé SOUS la VaR). Avec ARIMA, la bande de VaR est visuellement une bande quasi constante autour de la moyenne prévue, car σ ne varie pas dans le temps.

5. Volatilité : le modèle GARCH

Pourquoi GARCH ? Hétéroscédasticité conditionnelle

On a observé (chapitre 2) que la volatilité des rendements financiers n'est pas constante dans le temps : c'est de l'**hétéroscédasticité** (variance non constante), et plus précisément elle est **groupée** (une période volatile est suivie d'une autre période volatile). Le modèle **GARCH** (Generalized AutoRegressive Conditional Heteroskedasticity) modélise explicitement cette dynamique : la variance d'aujourd'hui dépend du carré du choc d'hier ET de la variance d'hier.

L'équation GARCH(1,1)

$$\sigma_t^2 = \omega + \alpha * \epsilon_{t-1}^2 + \beta * \sigma_{t-1}^2$$

- ω (>0) : variance de long terme (niveau de base) ;
- α : poids du choc récent au carré ϵ_{t-1}^2 -> capture la réaction immédiate à un choc (effet ARCH) ;
- β : poids de la variance passée σ_{t-1}^2 -> capture la persistance de la volatilité (mémoire longue de la variance) ;
- quand $\alpha + \beta$ est proche de 1, les chocs de volatilité sont très persistants - typique des marchés financiers.

Le code (cellule 22)

```
def walk_forward_garch(train, test, p=1, q=1):
    train_vals = pd.Series(np.asarray(train.values, dtype=float))
    am = arch_model(train_vals, mean="Constant", vol="GARCH", p=p, q=q, dist="normal")
    res = am.fit(dispatch="off")
    mu_c = float(res.params["mu"])
    omega = float(res.params["omega"])
    alpha = float(res.params["alpha[1]"])
    beta = float(res.params["beta[1]"])
    std_resid = np.asarray(res.std_resid.dropna())

    last_var = float(res.conditional_volatility.iloc[-1]) ** 2
    last_resid = float(train.values[-1] - mu_c)

    mu = np.full(len(test), mu_c)
    sigma = np.empty(len(test))
    for t in range(len(test)):
        var_t = omega + alpha * last_resid ** 2 + beta * last_var
        sigma[t] = np.sqrt(var_t)
        last_resid = float(test.values[t] - mu_c) # walk forward sur le realise
        last_var = var_t

    return ForecastResult(mu=mu, sigma=sigma, std_resid=std_resid,
                          y_true=test.values, dates=test.index, name=...)
```

Explication bloc par bloc

- `arch_model(..., mean="Constant", vol="GARCH", p=1, q=1, dist="normal")` : spécifie une moyenne constante (μ_c , pas d'ARIMA sur la moyenne ici) et une volatilité GARCH(1,1). Le modèle est ajusté **une seule fois** (`am.fit`) sur le train.
- On extrait les paramètres estimés une fois pour toutes : μ_c , ω , α , β , ainsi que le pool de résidus standardisés d'entraînement `std_resid` (utilisé plus tard pour le bootstrap de VaR).
- `last_var` et `last_resid` sont initialisés à partir de la **toute dernière observation du train** - c'est le point de départ de la récursion sur le test.
- **La boucle walk-forward** est la partie la plus importante : à chaque jour t , on calcule `var_t` avec la formule GARCH et les paramètres **fixes** (jamais réestimés). Puis `sigma[t]` est enregistré, et on met à jour `last_resid/last_var` avec le rendement **réellement réalisé** ce jour-là (`test.values[t]`) - c'est le

walk-forward sans réentraînement. Contrairement à ARIMA, ici σ **varie chaque jour** en fonction de la volatilité récente réellement observée.

- Cette technique - paramètres figés, mais variance recalculée jour après jour à partir du réalisé - s'appelle la **Simulation Historique Filtrée** (Filtered Historical Simulation, FHS) : on « filtre » la dynamique de variance du modèle tout en rééchantillonnant les résidus par bootstrap pour la forme de la distribution.

Résultats GARCH sur ADI (sortie réelle)

GARCH	MAE=0.6822	RMSE=0.9497	sigma moyen=0.9712
-------	------------	-------------	--------------------

Interprétation

Le MAE de GARCH (0.6822) est légèrement meilleur que celui d'ARIMA (0.7088) sur la prévision ponctuelle de la moyenne - mais l'apport réel de GARCH n'est pas là, il est dans le σ qui varie désormais jour après jour (sigma moyen 0.9712%, mais avec des hauts et des bas visibles sur le graphique de volatilité conditionnelle), contrairement au σ constant d'ARIMA. C'est cette dynamique qui rend la VaR GARCH plus réactive au risque réel du marché à chaque instant.

Contrairement à la VaR ARIMA (bande constante, cellule 19), la bande de VaR GARCH (cellule 23, à 99%) se resserre et s'élargit avec la volatilité réalisée - elle « respire » avec le marché, ce qui est exactement le comportement recherché pour une VaR réactive au risque.

6. Machine Learning : Random Forest et XGBoost

Approche supervisée par fenêtres de retards (lags)

Contrairement à ARIMA/GARCH (modèles économétriques paramétriques), Random Forest et XGBoost sont des modèles d'apprentissage supervisé génériques : ils ont besoin d'un tableau de **features (X)** et d'une **cible (y)**. On construit ce tableau en utilisant les rendements **retardés** (lags) comme features pour prédire le rendement du jour suivant.

```
def make_supervised(returns, n_lags):
    X, y = [], []
    for i in range(n_lags, len(returns)):
        X.append(returns[i - n_lags:i]); y.append(returns[i])
    return np.array(X), np.array(y)
```

Avec `n_lags=5`, chaque ligne de `X` contient les 5 derniers rendements `[r_(i-5), ..., r_(i-1)]`, et `y[i]` est le rendement à prédire `r_i`. C'est une transformation classique qui convertit un problème de série temporelle en un problème de régression tabulaire standard.

Les deux modèles (cellule 26)

```
def _model(kind):
    if kind == "rf":
        return RandomForestRegressor(n_estimators=300, random_state=SEED, n_jobs=-1)
    if kind == "xgb":
        return XGBRegressor(n_estimators=300, max_depth=4, learning_rate=0.05,
                             random_state=SEED, n_jobs=-1)

def walk_forward_ml(train, test, model_kind, n_lags=5):
    Xtr, ytr = make_supervised(train.values, n_lags)
    model = _model(model_kind).fit(Xtr, ytr)
    resid = ytr - model.predict(Xtr)
    sigma = float(np.std(resid))
    std_resid = (resid - resid.mean()) / sigma
    hist = list(train.values[-n_lags:])
    mu = np.empty(len(test))
    for t in range(len(test)):
        mu[t] = float(model.predict(np.array(hist[-n_lags:]).reshape(1, -1))[0])
        hist.append(test.values[t]) # on réinjecte le rendement réalisé
    return ForecastResult(mu=mu, sigma=np.full(len(test), sigma), std_resid=std_resid, ...)
```

Explication

- **Random Forest** : ensemble de 300 arbres de décision (`n_estimators=300`), chacun entraîné sur un sous-échantillon aléatoire des données et des features ; la prévision finale est la moyenne des 300 arbres. Réduit le surapprentissage d'un arbre unique.
- **XGBoost** : ensemble de 300 arbres construits **séquentiellement** (boosting), chaque nouvel arbre corrigeant les erreurs des précédents, avec un taux d'apprentissage faible (`learning_rate=0.05`) qui limite l'influence de chaque arbre individuel et une profondeur limitée (`max_depth=4`) qui limite la complexité de chaque arbre.
- Le modèle est **entraîné une seule fois** sur (`Xtr, ytr`) (train uniquement). Les résidus d'entraînement (`resid = ytr - prédictions`) donnent `sigma` et le pool de résidus standardisés.
- Dans la boucle `walk-forward`, `hist` accumule l'historique des rendements ; à chaque pas, on prédit à partir des 5 derniers rendements connus (`hist[-n_lags:]`), puis on ajoute le rendement **réellement réalisé** ce jour-là à `hist` (jamais de réentraînement du modèle).

Résultats RF / XGBoost sur ADI (sortie réelle)

Random Forest	MAE=0.6945	RMSE=0.9757	sigma=0.4731
XGBoost	MAE=0.6656	RMSE=0.9064	sigma=0.8125

Interprétation - attention, piège classique

Les deux modèles ML obtiennent un MAE/RMSE comparable, voire meilleur pour XGBoost (0.6656), aux modèles classiques sur ADI - à première vue, ce sont de bons modèles de prévision ponctuelle. MAIS on verra aux chapitres 8-10 qu'un bon score de prévision ponctuelle (MAE/RMSE) ne garantit absolument pas une bonne calibration de la VaR : Random Forest, en particulier, s'avère nettement sur-violateur au backtesting (zone rouge Bâle), malgré un MAE tout à fait correct. La raison technique : le sigma de RF (0.4731) est ici anormalement faible par rapport à celui des autres modèles (proche de 1.0) - Random Forest a tendance à produire des prévisions « mean-reverting » trop lisses, ce qui sous-estime la variabilité réelle et donc rétrécit excessivement la bande de VaR.

7. Deep Learning : ANN et LSTM

Réseau feedforward (ANN) vs réseau récurrent (LSTM)

Un **ANN** (perceptron multicouche, feedforward) traite un vecteur d'entrée fixe (ici les 10 derniers rendements) en le faisant passer à travers des couches de neurones successives, sans aucune notion explicite d'ordre temporel entre les entrées : chaque entrée est juste une coordonnée parmi d'autres.

Un **LSTM** (Long Short-Term Memory) est un réseau **récurrent** spécialement conçu pour les séquences : il traite les rendements un par un, dans l'ordre, et maintient un **état interne** (mémoire) qui se met à jour à chaque pas de temps via des **portes** (gate d'entrée, gate d'oubli, gate de sortie) qui contrôlent quelle information est conservée, oubliée, ou utilisée pour la sortie. Cela lui permet en théorie de capturer des dépendances temporelles plus riches qu'un ANN, y compris à moyen terme.

Le code des architectures (cellule 29)

```
class _ANN(nn.Module):
    def __init__(self, w):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(w, 32), nn.ReLU(),
            nn.Linear(32, 16), nn.ReLU(),
            nn.Linear(16, 1),
        )
    def forward(self, x):
        return self.net(x).squeeze(-1)

class _LSTM(nn.Module):
    def __init__(self, w):
        super().__init__()
        self.lstm = nn.LSTM(1, 32, batch_first=True)
        self.fc = nn.Linear(32, 1)
    def forward(self, x):
        o, _ = self.lstm(x.unsqueeze(-1))
        return self.fc(o[:, -1, :]).squeeze(-1)
```

- **_ANN** : 3 couches linéaires (w -> 32 -> 16 -> 1) séparées par des activations ReLU (non-linéarités qui permettent au réseau d'apprendre des relations non linéaires) ; w est la taille de la fenêtre d'entrée (10 rendements passés).
- **_LSTM** : une couche nn.LSTM(1, 32, batch_first=True) (1 valeur par pas de temps, 32 unités de mémoire cachée), suivie d'une couche linéaire finale 32 -> 1. o[:, -1, :] prend uniquement la sortie du **dernier pas de temps** de la séquence (résumé de toute la séquence traitée) pour produire la prévision.

Entraînement et walk-forward (cellule 29, suite)

```
def _train(model, X, y, epochs):
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    loss = nn.MSELoss()
    Xt, yt = torch.tensor(X, dtype=torch.float32), torch.tensor(y, dtype=torch.float32)
    for _ in range(epochs):
        opt.zero_grad(); l = loss(model(Xt), yt); l.backward(); opt.step()
    return model

def walk_forward_dl(train, test, kind, window=10, epochs=30):
    mu_, sd_ = train.mean(), train.std()
    z = (train.values - mu_) / sd_ # standardisation
    X, y = make_sequences(z, window)
    model = _ANN(window) if kind == "ann" else _LSTM(window)
    model = _train(model, X, y, epochs)
    ...
    hist = list(z[-window:])
    for t in range(len(test)):
        pred_z = model(hist[-window:])
```

```
mu[t] = pred_z * sd_ + mu_      # de-standardisation
hist.append((test.values[t] - mu_) / sd_)  # reinjection du realise (standardise)
```

Explication

- $z = (\text{train.values} - \mu_) / \text{sd_}$: **standardisation** (moyenne 0, écart-type 1) des rendements avant de les fournir au réseau - une étape standard en deep learning car elle stabilise et accélère l'optimisation par descente de gradient (Adam).
- `_train` : boucle d'apprentissage classique - `opt.zero_grad()` réinitialise les gradients, `loss(model(Xt), yt)` calcule l'erreur quadratique moyenne (MSE) entre prévision et cible, `.backward()` calcule les gradients par rétropropagation, `opt.step()` met à jour les poids. Répété pendant `epochs` itérations sur l'ensemble d'entraînement (**20 époques** dans ce notebook, contre 30 dans le run de référence du package, pour réduire le temps d'exécution).
- Le modèle est entraîné **une seule fois**, puis avancé pas à pas sur le test : à chaque jour, on prédit `pred_z` (standardisé), on le « dé-standardise » ($\text{pred_z} * \text{sd_} + \mu_$) pour obtenir la prévision en pourcentage réel, puis on réinjecte le rendement **réellement réalisé** (re-standardisé) dans l'historique `hist` pour la prochaine prévision - encore une fois, jamais de réentraînement.

Résultats ANN / LSTM sur ADI (sortie réelle)

ANN	MAE=0.6774	RMSE=0.9418
LSTM	MAE=0.6809	RMSE=0.9469

Interprétation - premier indice sur l'hypothèse du projet

Sur ADI, l'ANN (MAE 0.6774) et le LSTM (MAE 0.6809) obtiennent des scores de prévision très proches l'un de l'autre, et proches des modèles classiques (ARIMA 0.7088, GARCH 0.6822). Rien, à ce stade, ne suggère une supériorité franche du LSTM sur l'ANN ou sur les modèles classiques. On affine ce jugement avec le backtesting de la VaR (chapitres 8-10), qui est le vrai critère de décision de ce projet - la MAE/RMSE de prévision ponctuelle n'est qu'un indice préliminaire.

8. La VaR par Bootstrap Historical Simulation (BHS)

Rappel de la formule et du rôle de chaque terme

$$\text{VaR}_t(\alpha) = \mu_t + \sigma_t * Q_{\alpha}(\text{bootstrap}(\text{residus_standardises_train}))$$

- μ_t, σ_t : prévision de moyenne / écart-type du modèle au jour t du test - **constants** pour ARIMA, RF, XGB, ANN, LSTM (un seul σ calculé une fois sur le train), **dynamiques** pour GARCH (recalculé chaque jour, chapitre 5).
- $Q_{\alpha}(\dots)$: quantile empirique α (queue gauche, valeur typiquement négative) d'un rééchantillonnage bootstrap (avec remise, 10 000 tirages) du pool de résidus standardisés de l'entraînement.
- Une **violation** est un jour où le rendement réellement réalisé est **strictement inférieur** à la VaR ($y_{true} < \text{VaR}$) - c'est-à-dire un jour où la perte réelle a dépassé le niveau que la VaR prétendait ne pas devoir dépasser plus de α fois sur 100.

Pourquoi un bootstrap et pas une hypothèse de loi normale ? Parce que les résidus des modèles financiers ne suivent généralement pas une loi normale (kurtosis élevée, chapitre 1) : utiliser directement le quantile empirique des résidus (via rééchantillonnage) capture la vraie forme de la distribution (queues épaisses comprises) sans imposer d'hypothèse paramétrique potentiellement fausse.

Illustration : VaR LSTM à 95% sur ADI (cellules 32-33)

```
var_lstm_95 = var_series(fc_lstm, 0.05)
n_viol = int((fc_lstm.y_true < var_lstm_95).sum())
print(f"LSTM @95% sur ADI: {n_viol} violations / {len(var_lstm_95)} jours "
      f"(taux observe = {n_viol/len(var_lstm_95):.4f}, attendu = 0.05)")
```

```
LSTM @95% sur ADI: 3 violations / 170 jours (taux observe = 0.0176, attendu = 0.05)
```

Interprétation

Sur les 170 jours du test ADI, seulement 3 violations sont observées à un seuil théorique de 95% (soit un taux observé de 1.76%, contre 5% attendu). Le modèle LSTM est donc ici plutôt conservateur (il sous-estime légèrement le risque de violation, ce qui est préférable à l'inverse d'un point de vue prudentiel, mais peut aussi signaler une VaR un peu trop large). Le chapitre suivant (Kupiec/Christoffersen) donne les outils statistiques pour juger précisément si cet écart de 1.76% vs 5% est significatif ou seulement du bruit d'échantillonnage.

9. Backtesting : Kupiec, Christoffersen, zones Bâle

Pourquoi backtester une VaR ?

Calculer une VaR ne suffit pas : il faut vérifier, a posteriori, qu'elle est **bien calibrée** - que le taux de violations réellement observé est statistiquement cohérent avec le taux attendu (α). C'est le rôle du **backtesting**, avec deux tests complémentaires et une classification réglementaire.

Test de Kupiec (POF - Proportion of Failures)

H0 : le taux de violation observé est statistiquement compatible avec α (bonne couverture non-conditionnelle). Le test compare, via un rapport de vraisemblance (likelihood ratio), la probabilité d'observer n_{viol} violations sur n_{obs} jours sous l'hypothèse α théorique versus sous le taux observé $p_i = n_{\text{viol}}/n_{\text{obs}}$.

```
def _safe_log(x):
    return np.log(x) if x > 0 else 0.0

def kupiec_pof(n_viol, n_obs, alpha):
    pi = n_viol / n_obs
    ll_null = (n_obs - n_viol) * _safe_log(1 - alpha) + n_viol * _safe_log(alpha)
    ll_alt = (n_obs - n_viol) * _safe_log(1 - pi) + n_viol * _safe_log(pi)
    lr = -2 * (ll_null - ll_alt)
    p = 1 - stats.chi2.cdf(lr, 1)
    return {"LR": lr, "pvalue": p, "reject": bool(p < 0.05)}
```

- `_safe_log(x)` : variante « protégée » du logarithme qui renvoie 0 si $x \leq 0$, pour éviter un $\log(0) = -\infty$ dans les cas limites $n_{\text{viol}} = 0$ ou $n_{\text{viol}} = n_{\text{obs}}$ (aucune ou toutes les observations sont des violations).
- `ll_null` : log-vraisemblance des données **sous l'hypothèse que le vrai taux est α** .
- `ll_alt` : log-vraisemblance des données **sous le taux effectivement observé π** (l'ajustement « parfait » aux données).
- `lr = -2 * (ll_null - ll_alt)` : statistique de rapport de vraisemblance, qui suit asymptotiquement une loi du Khi-2 à 1 degré de liberté sous H0.
- **Lecture** : $p < 0.05 \rightarrow$ on rejette H0 $\rightarrow$ le modèle est **mal calibré** (trop ou pas assez de violations par rapport à ce qui est attendu). p élevé $\rightarrow$ bonne calibration.

Test de Christoffersen (couverture conditionnelle)

Le test de Kupiec vérifie seulement le **nombre total** de violations, pas leur répartition dans le temps. Or une VaR bien calibrée devrait produire des violations **indépendantes** les unes des autres (pas groupées). Le test de Christoffersen ajoute un test d'**indépendance** des violations à celui de Kupiec, pour obtenir un test de **couverture conditionnelle**.

```
def christoffersen(viol, alpha):
    v = np.asarray(viol).astype(int)
    n00 = n01 = n10 = n11 = 0
    for a, b in zip(v[:-1], v[1:]):
        if a == 0 and b == 0: n00 += 1
        elif a == 0 and b == 1: n01 += 1
        elif a == 1 and b == 0: n10 += 1
        else: n11 += 1
    pi = (n01 + n11) / max(n00 + n01 + n10 + n11, 1)
    pi0 = n01 / max(n00 + n01, 1); pi1 = n11 / max(n10 + n11, 1)
    ll_ind = (n00 + n10) * _safe_log(1 - pi) + (n01 + n11) * _safe_log(pi)
    ll_alt = n00*_safe_log(1-pi0) + n01*_safe_log(pi0) + n10*_safe_log(1-pi1) + n11*_safe_log(pi1)
    lr_ind = -2 * (ll_ind - ll_alt)
    lr_cc = kupiec_pof(int(v.sum()), len(v), alpha)["LR"] + lr_ind
    p_cc = 1 - stats.chi2.cdf(lr_cc, 2)
    return {"LR_ind": lr_ind, "LR_cc": lr_cc, "pvalue_cc": p_cc, "reject": bool(p_cc < 0.05)}
```

- n_{00} , n_{01} , n_{10} , n_{11} : compte les transitions entre jours consécutifs - n_{01} = jour sans violation suivi d'un jour avec violation, n_{11} = deux jours de violation consécutifs, etc. On compare π_0 (probabilité de violation sachant que la veille n'en était pas une) à π_1 (probabilité de violation sachant que la veille en était déjà une) : si les violations sont indépendantes, π_0 devrait être proche de π_1 .
- lr_{cc} = LR_{Kupiec} + $LR_{indépendance}$: la statistique de couverture conditionnelle **combine** les deux effets (proportion correcte ET indépendance), et suit une loi du Khi-2 à 2 degrés de liberté (au lieu de 1 pour Kupiec seul).
- **Lecture** : $p_{cc} < 0.05 \rightarrow$ rejet $\rightarrow$ soit le taux de violation est incorrect, soit les violations sont groupées dans le temps (ou les deux).

Zones de Bâle (green / yellow / red)

Classification réglementaire (accords de Bâle sur les fonds propres bancaires) du nombre de violations, normalisé sur une fenêtre standard de 250 jours de bourse (environ un an) :

```
def basel_zone(n_viol, n_obs, alpha):
    scaled = n_viol * (250 / n_obs)
    if alpha == 0.01:
        return "green" if scaled <= 4 else "yellow" if scaled <= 9 else "red"
    return "green" if scaled <= 17 else "yellow" if scaled <= 25 else "red"
```

- **Zone verte** : nombre de violations conforme aux attentes statistiques $\rightarrow$ aucune action requise, le modèle interne peut être utilisé tel quel pour le calcul des fonds propres.
- **Zone jaune** : zone d'alerte $\rightarrow$ le régulateur peut demander des justifications ou augmenter le facteur multiplicatif appliqué à la VaR réglementaire.
- **Zone rouge** : nombre de violations trop élevé, incompatible avec une bonne calibration $\rightarrow$ le modèle interne est remis en cause par le régulateur.

Application aux 7 modèles sur ADI (tableau réel, alpha = 5% et 1%)

Modèle	alpha	MAE	RMSE	Taux viol.(%)	p Kupiec	p Christ.	Zone Bâle
ARIMA	0.05	0.71	0.98	1.76	0.0263	0.0817	vert
ARIMA	0.01	0.71	0.98	0.59	0.5588	0.8380	vert
SARIMA	0.05	0.71	0.98	1.76	0.0263	0.0817	vert
SARIMA	0.01	0.71	0.98	0.59	0.5588	0.8380	vert
GARCH	0.05	0.68	0.95	5.88	0.6071	0.4670	vert
GARCH	0.01	0.68	0.95	1.18	0.8220	0.9519	vert
RF	0.05	0.69	0.98	21.76	1.3e-13	2.6e-13	rouge
RF	0.01	0.69	0.98	3.53	0.0099	0.0127	jaune
XGB	0.05	0.67	0.91	5.29	0.8616	0.7135	vert
XGB	0.01	0.67	0.91	0.59	0.5588	0.8380	vert

Modèle	alpha	MAE	RMSE	Taux viol.(%)	p Kupiec	p Christ.	Zone Bâle
ANN	0.05	0.68	0.94	1.18	0.0063	0.0233	vert
ANN	0.01	0.68	0.94	0.59	0.5588	0.8380	vert
LSTM	0.05	0.68	0.95	1.76	0.0263	0.0803	vert
LSTM	0.01	0.68	0.95	0.59	0.5588	0.8380	vert

Interprétation (ADI, résultat réel du notebook)

À 99%, presque tous les modèles sont en zone verte avec un taux observé (0.59% à 1.18%) proche du taux attendu (1%) et des p-values de Kupiec élevées (0.56 à 0.82) : ARIMA, SARIMA, GARCH, XGB, ANN, LSTM sont tous bien calibrés sur ADI à ce seuil. Random Forest est le seul modèle problématique : à 95%, son taux de violation observé explose à 21.76% (contre 5% attendu), avec une p-value de Kupiec quasi nulle ($1.3e-13$) -> rejet massif -> zone rouge. Même à 99%, RF reste en zone jaune (3.53% observé contre 1% attendu, $p=0.0099$, proche du seuil de rejet 0.05). Ce tableau confirme au niveau ADI le problème déjà pressenti au chapitre 6 : un bon MAE ponctuel (RF avait un MAE tout à fait correct) ne garantit pas une VaR bien calibrée. Le détail exact des « gagnants » par indice est traité au chapitre 10 avec les 4 indices MENA.

10. Comparaison globale (4 indices x 7 modèles) et conclusion

Assemblage du pipeline complet (cellule 38)

```
MODELS = {
    "ARIMA": lambda tr, te: walk_forward_arma(tr, te),
    "SARIMA": lambda tr, te: walk_forward_sarima(tr, te, m=5),
    "GARCH": lambda tr, te: walk_forward_garch(tr, te),
    "RF": lambda tr, te: walk_forward_ml(tr, te, "rf"),
    "XGB": lambda tr, te: walk_forward_ml(tr, te, "xgb"),
    "ANN": lambda tr, te: walk_forward_dl(tr, te, "ann", epochs=20),
    "LSTM": lambda tr, te: walk_forward_dl(tr, te, "lstm", epochs=20),
}

def run_index(name, data_dir, models=None, alphas=(0.05, 0.01), cache=None):
    tr, te = train_test_returns(name, data_dir)
    rows = []
    for mname in (models or list(MODELS)):
        fc = cache.setdefault(mname, MODELS[mname](tr, te))
        ... # MAE, RMSE, puis var_series + backtest_summary pour chaque alpha
    return pd.DataFrame(rows)

def run_all(data_dir, indices=("Tunindex", "ADI", "MASI", "TASI"), caches=None):
    return pd.concat([run_index(i, data_dir, cache=caches.get(i)) for i in indices],
                     ignore_index=True)

results = run_all(DATA_DIR, caches={"ADI": adi_forecasts})
```

Explication

- MODELS : dictionnaire regroupant les 7 « forecasters » définis dans les chapitres précédents sous une interface commune (train, test) -> ForecastResult.
- run_index : pour un indice donné, ajuste les 7 modèles (ou réutilise le cache s'ils ont déjà été calculés, cas d'ADI dont les forecasts des chapitres 4-7 sont réutilisés tels quels pour éviter un calcul redondant) et backteste la VaR aux 2 seuils alpha -> 14 lignes par indice.
- run_all : boucle sur les 4 indices MENA (Tunindex, ADI, MASI, TASI) et concatène le tout -> **4 x 7 x 2 = 56 lignes** au total. C'est le calcul le plus long du notebook (plusieurs minutes), car Tunindex, MASI et TASI sont calculés ici pour la première fois (ADI était déjà fait).

Le tableau des « meilleurs modèles » par indice (cellule 39, résultat réel)

```
best = (results[results.alpha == 0.01]
        .sort_values("kupiec_p", ascending=False)
        .groupby("index").first().reset_index())
```

Pour chaque indice, on trie les modèles par p-value de Kupiec **décroissante** (plus elle est élevée, plus le taux de violation observé est proche de l'attendu, donc plus le modèle est bien calibré) et on garde le premier -> le « meilleur » modèle au sens de la calibration VaR à 99%.

Indice	Meilleur modèle	Taux observé (1% attendu)	p Kupiec	Zone Bâle
ADI	GARCH	1.18%	0.8220	green
MASI	GARCH	0.63%	0.6138	green
TASI	ARIMA	0.60%	0.5734	green
Tunindex	GARCH	1.26%	0.7534	green

Interprétation - le résultat central du projet

GARCH est le meilleur modèle (au sens de la p-value de Kupiec la plus élevée, à $\alpha=1\%$) sur 3 des 4 indices MENA : ADI, MASI et Tunindex. Sur TASI, c'est ARIMA qui l'emporte de justesse. GARCH gagne donc la majorité (3/4) des indices : son avantage - une variance qui s'adapte jour après jour au clustering de volatilité réel du marché (chapitre 5) - se traduit concrètement par une meilleure calibration de la VaR sur la plupart des indices MENA étudiés.

Taux de violation observé par modèle x indice, à 99% (cellule 41, résultat réel)

Modèle	Tunindex	ADI	MASI	TASI
ANN	0.63	0.59	0.00	0.60
ARIMA	0.63	0.59	0.00	0.60
GARCH	1.26	1.18	0.63	1.80
LSTM	0.63	0.59	0.00	0.60
RF	6.29	3.53	4.40	4.79
SARIMA	0.63	0.59	0.00	0.60
XGB	1.89	0.59	0.63	1.80

Interprétation

Le taux attendu est de 1% (seuil $\alpha=1\%$). Random Forest (ligne RF) affiche systématiquement les taux de violation les plus élevés sur les 4 indices (de 3.53% à 6.29%, soit 3.5 à 6 fois le taux attendu) : c'est une confirmation, à l'échelle des 4 indices, du problème déjà identifié sur ADI seul (chapitre 9) - Random Forest sur-viole de manière structurelle et n'est pas adapté à un usage de VaR réglementaire. À l'inverse, ANN, ARIMA, LSTM et SARIMA affichent des taux très proches ou légèrement inférieurs à 1% sur les 4 indices - une calibration très correcte. GARCH affiche des taux légèrement supérieurs à 1% (jusqu'à 1.80% sur TASI) mais reste globalement bien calibré (voir les p-values de Kupiec, chapitre 9).

Verdict de l'hypothèse : « le LSTM surpasse sur ADI »

Rappel de l'hypothèse de départ (chapitre 0) : « **le LSTM surpasse les autres modèles sur l'indice ADI** ». D'après le tableau best ci-dessus, le meilleur modèle sur ADI (au sens de la p-value de Kupiec à 99%) est **GARCH** ($p=0.8220$), pas le LSTM. Le détail du chapitre 9 montre que le LSTM est **bien calibré** sur ADI (zone verte aux deux seuils, p-values 0.03 à 0.56 selon l' α) - ce n'est donc pas un mauvais modèle - mais il n'est **pas distinctement supérieur** : GARCH, ARIMA, SARIMA, XGB et ANN atteignent une calibration du même ordre, voire meilleure.

Conclusion honnête sur l'hypothèse

L'hypothèse de départ n'est PAS confirmée par ce run. Le LSTM fait partie des modèles fiables sur ADI, sans en être le meilleur. C'est un résultat scientifique à assumer tel quel à l'oral : il ne s'agit pas d'un échec du projet, mais d'une conclusion empirique légitime, qui illustre d'ailleurs un point pédagogique important (voir plus bas : pourquoi le deep learning ne domine pas nécessairement sur ce type de données).

Conclusion générale : quel modèle pour la VaR MENA ?

- **GARCH est le modèle le plus robuste dans l'ensemble** : sa VaR dynamique, qui « respire » avec la volatilité réalisée, est cohérente avec le clustering de volatilité observé dès le chapitre 2 - un modèle qui module explicitement σ_t dans le temps capture mieux le risque de queue qu'un modèle à variance constante. Il gagne 3 des 4 indices MENA (ADI, MASI, Tunindex).

- **ARIMA/SARIMA restent des VaR solides et peu coûteuses** en temps de calcul, avec un sigma constant qui suffit néanmoins souvent à passer les tests de calibration - ARIMA gagne même sur TASI.
- **Random Forest est à écarter pour la VaR** : il sur-viole nettement et de façon constante sur les 4 indices (souvent zone rouge ou jaune, Kupiec rejeté ou proche du rejet) - un bon MAE ponctuel ne suffit pas à produire une VaR bien calibrée ; son pool de résidus, combiné à un forecast trop « mean-reverting », produit des bandes de VaR trop étroites.
- **Les modèles Deep Learning (ANN, LSTM) sont honorables mais pas transformateurs** : généralement bien calibrés, du même ordre que les modèles classiques, sans avantage net qui justifierait leur coût d'entraînement supplémentaire sur ces échantillons - d'autant que ce notebook les entraîne avec un budget d'époques réduit (20) par souci de temps d'exécution.

Pourquoi le deep learning ne domine-t-il pas ici ?

C'est une question probable à l'oral. Deux raisons principales, à bien maîtriser :

- les échantillons MENA sont **de petite taille** (quelques centaines à un peu plus de 2500 observations d'entraînement selon l'indice) - très modeste pour entraîner efficacement un LSTM, qui a généralement besoin de beaucoup de données pour exploiter pleinement sa capacité à modéliser des dépendances complexes ;
- la dynamique de volatilité des rendements financiers est **fortement structurée** (clustering, autocorrélation de la variance) - exactement ce que GARCH est spécifiquement conçu pour modéliser de façon paramétrique et parcimonieuse (seulement 3 paramètres : omega, alpha, beta). Un réseau de neurones doit *apprendre* cette structure à partir de peu de données, sans aucun a priori, ce qui limite son avantage face à un modèle économétrique dédié et spécialisé pour ce type de dynamique.

Sur des échantillons plus longs ou à plus haute fréquence (données intra-journalières, par exemple), l'avantage du deep learning pourrait être différent - c'est une piste d'amélioration à mentionner à l'oral si on vous demande « comment améliorer le projet ».

Bilan méthodologique

Le point le plus important à retenir pour la soutenance

Ce projet illustre un point clé en gestion des risques : la performance de prévision ponctuelle (MAE/RMSE) et la qualité de calibration de la VaR (Kupiec/Christoffersen) sont deux choses différentes. Random Forest le montre parfaitement : son MAE est souvent correct (voire meilleur que ARIMA sur certains indices), mais sa VaR est très mal calibrée. Pour un usage réglementaire (Bâle), c'est la calibration de la QUEUE de distribution qui prime sur la précision du point médian - ce qui favorise structurellement les modèles qui modélisent explicitement la variance dans le temps (GARCH) sur les indices MENA étudiés ici.